NETSUITE ASSIGNMENT - USER EVENT SCRIPT

1. What are User Event Scripts?

Ans.

- "User Event Scripts" that are triggered in response to user-initiated events within the system. These scripts are designed to respond to specific actions or changes made by users.
- This is Server-Side script.
- When Server-side script get trigger -
  1. View any record
  2. Creating a new record
  3. Updating an existing record
  4. Saving a record
- We have 3 events in user event script -
  1. BeforeLoad() - This event trigger before record is loaded
  2. BeforeSubmit() - This event trigger before record gets save to database
  3. AfterSubmit() - This event trigger after record is submitted
- Navigation to create new user event script -
- Customization > Script> New > Upload user event script file with .js extension > Create script record > Enter file name and Id > Save > Deploy Script > Applies to record where we want to trigger this script > Select status as 'Testing' > Execute as a role 'Administrator' > Log level 'debug' > Save

2. What is the purpose of User Event Scripts?

Ans.

Purpose of User Event Scripts -

Data Validation:

- We can use user events to validate data entered by users.
- For example, to ensure that certain fields contain valid information before allowing a record to be saved.
- In this case, we can use beforeSubmit event to validate field

Custom Business Logic:

- User events allow us to implement custom business logic or workflows that are specific to an organization's needs.
- This can include automating processes, calculations, or generating custom notifications.

- For example, we want to allow employee center to view purchase order if it is approved by Purchase manager else employee center won't be able to view that purchase record
- In this case we could use beforeLoad event to check the status of purchase order if it approved, employee center will be able to open purchase order.

Record Updates:

- We can use user events to automatically update records or related records based on changes made to a specific record.
- For example, when a sales order is approved, we can update inventory levels and create corresponding purchase orders.
- In this case, we can use afterSubmit event to update inventory levels once stock gets dispatch or receive.

Security and Permissions:

- User events can be used to enforce security and permissions. You can restrict access or actions based on specific criteria, ensuring that only authorized users can perform certain actions.
- For example, we can restrict users to edit or create any new record, Employee center will not be able to create new purchase order they only can create purchase requisition.
- Hence, here we will use beforeLoad event to check permission of the user if it is Administrator then we will allow him to create new purchase order and if it is employee center, he could only view purchase order.

Field Calculations:

- User events are useful for automatically calculating and populating fields based on predefined rules or formulas.

Integration:

- User events can be integrated with external systems and services by triggering events when certain conditions are met.
- For example, once order has been place, we can redirect user to payment gateways
- In this case we can use aftersubmit event, once order request gets saved on database, user will redirect to payement.

3. What are the user event script functions?

Ans.

There are 3 user event scripts function:

beforeLoad:

- This script type is executed when a record is being loaded or viewed, but before it is displayed to the user.
- "beforeLoad" can use scripts to manipulate the record data, change the form's appearance, or perform user authorization and validation before the user interacts with the record, to hide/show fields, set field default values, or add client-side scripting logic to enhance the user experience when viewing or editing records.
- Declaration of event: function beforeLoad(context) {}
- This event has “Void” return type.
- Parameters of event:
  1. form: the object of current form
  2. newRecord: The new record (the record being loaded).
  3. request: The HTTP request information sent by the browser.
  4. type: The type of operation invoked by the event (the trigger type).

beforeSubmit:

- This script type is triggered before the record is submitted to the server but after a user has made changes to the record and clicks "Save" or "Submit." It is used for validation and data manipulation before the record is saved to the database.
- "beforeSubmit" scripts are often used for data validation, calculation, modification of record data before it's saved or prevent certain types of records from being saved.
- Basically, this event is used to trigger for status like approve, reject, cancel, pack, mark complete, reassign and ship etc.
- Declaration of event: function beforeSubmit(context) {}
- This event has “Void” return type.
- Parameters of event:
  1. newRecord: The new (or updated) record.
  2. oldRecord: The old record (the previous state of the record).
  3. type: The type of operation invoked by the event (the trigger type).

afterSubmit:

- This script type is triggered after the record has been successfully saved to the database. It is used for performing actions or sending notifications after the record has been committed to the database.
- "afterSubmit" scripts are used for additional data processing or integration tasks, such as sending notifications, updating related records, triggering external processes or performing other actions that depend on the data being successfully saved.
- Basically, this event is used to trigger for status like approve, reject, cancel, pack, dropship, paybills , orderitem and ship etc.
- Declaration of event: function afterSubmit(context) {}
- This event has "Void" return type.
- Parameters of event:
    4. newRecord: The new (or updated) record in read-only mode.
    5. oldRecord: The old record (previous state of the record) in read-only mode.
    6. type: The type of operation invoked by the event (the trigger type).

4. Can we call a user event script from other scripts?

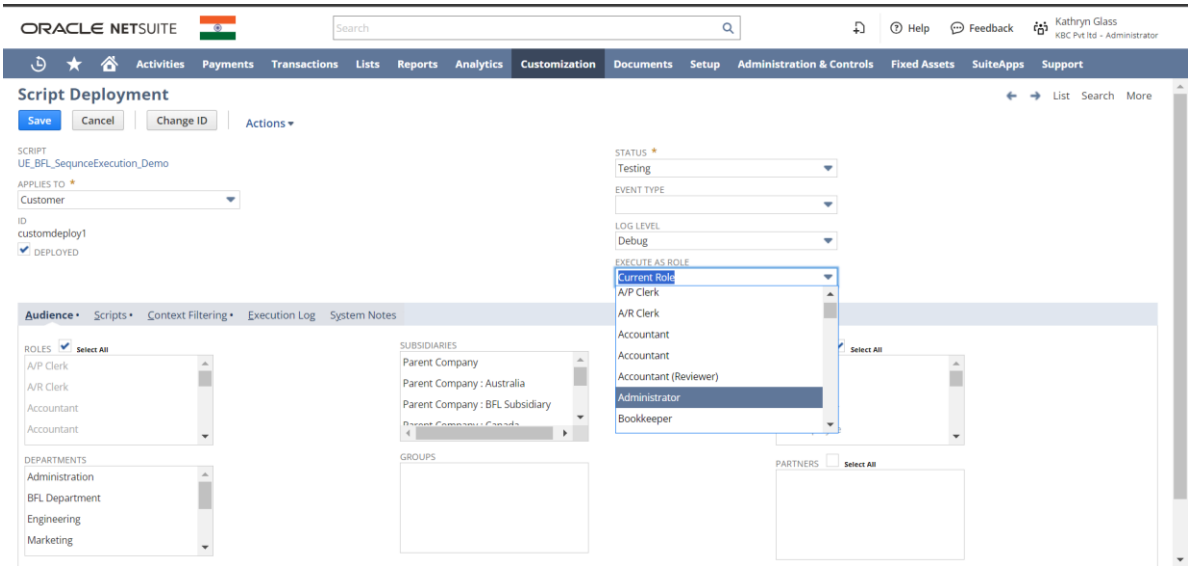
Ans.

- User event scripts cannot be called from another user event script, or by workflows with a Context of User Event Script and it also doesn't trigger when you submit a custom record via other scripts.
- User Event Before Submit or After Submit functions can be triggered from other scripts by submitting the record on which user event script is deployed.
- We can execute a user event script from a call within-
    1. Scheduled script
    2. Portlet script
    3. Suitelet script

5. Can we execute user event script as admin role?

Ans.

- Yes, we can execute user event script as admin role in script deployment.
- Navigation to execute user event script as admin role -
- Customization > Script > New > Upload user event script file with .js extension > Create script record > Enter file name and Id > Save > Deploy Script > Execute as a role 'Administrator'



6. What is the governance limit for user event scripts?

Ans. Governance limit for user event scripts is 1000 units.

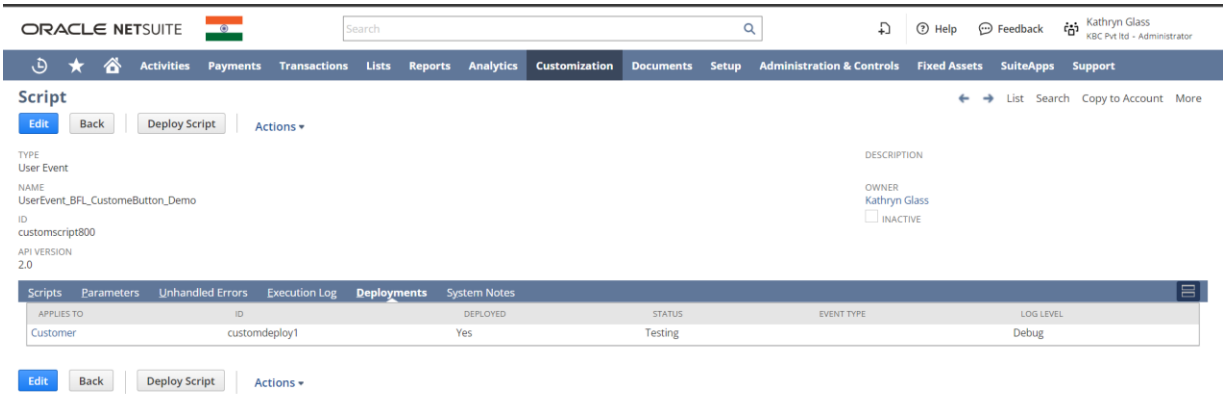
7. How can we create custom button using user event script?

Ans.

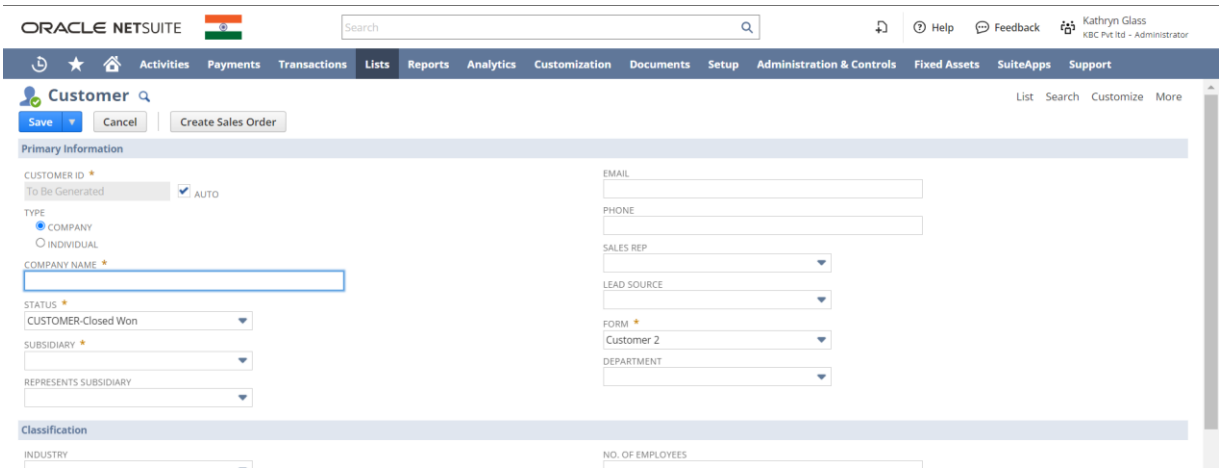
- In User Event script we can use beforLoad event to add custom button on customer page.
- In beforLoad event we will use form.addButton({}) event to add new button
- Labe of button will be ‘Create sales order’.

```
/**
 * @NApiVersion 2.1
 * @NScriptType UserEventScript
 */
define(["N/record", "N/log"], (record, log) => {
  const beforeLoad = (scriptContext) => {
    var form = scriptContext.form;
    form.addButton({
      id: "custpage_custombutton",
      label: "Create Sales Order",
    });


  };

  return { beforeLoad};
});
```

- Script has deployed on customer record



- ‘Create sales order’ button has been added on customer form



8. How can we prevent users from submitting a record?

Ans.

- We can use client script to prevent user to submit record
- Here we will return false in saveRecord() which will not allow user to save any record.

```
define(["N/record"],

function(record) {

    function pageInit(scriptContext) {
        log.debug({
        title: "pageInit"
    });
    }


    function saveRecord(scriptContext) {
      alert('test save record');
        log.debug({
        title: "saveRecord "
         });
      return false;
    }

    return {
        pageInit: pageInit,
        saveRecord: saveRecord
    };

});
```

9. Is there any limit for the number of user event scripts that can be deployed in one record?

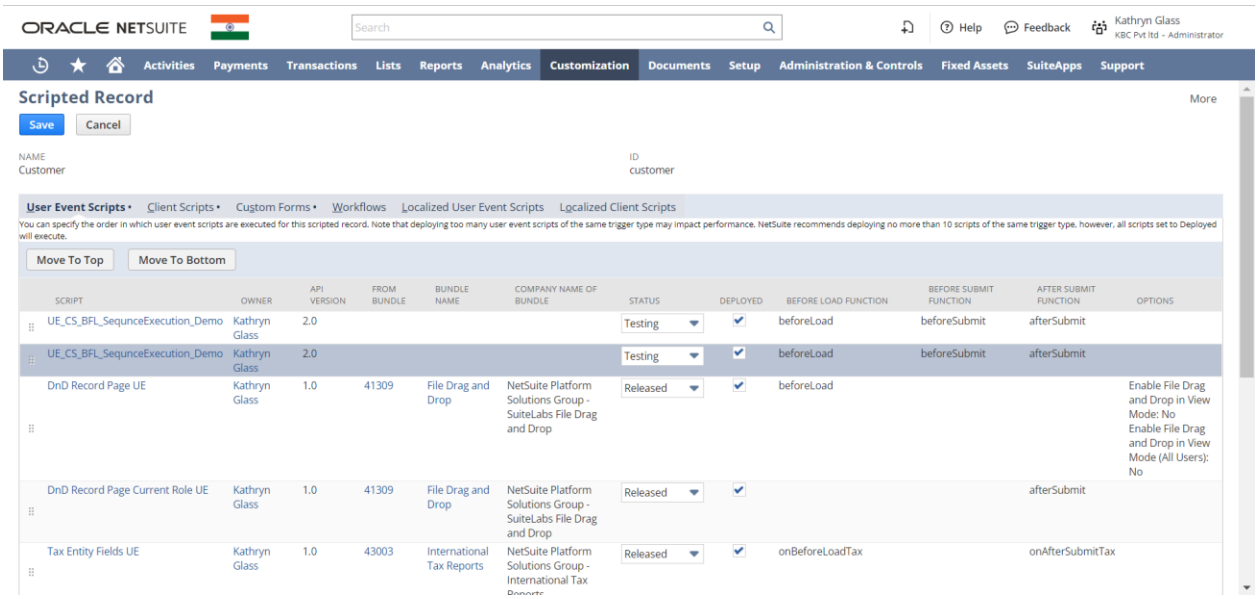
Ans.

- There is no limit on the number of client or user event scripts that can be deployed.
- However, performance issues may occur if we have a lot of scripts deployed on a single record.

10. How to execute one user event script after or before another?

Ans.

- Scripted Records can be used to place the sequence in which user event scrip needs to be executed.
- Navigation to modify execution of user event scripts:
- Customization > Scripting > Scripted records > Edit the entity/transaction record on which our user event script > User event script > Drag and Drop the script in order of execution
- Likewise, we can set order of execution of all script of suite script 2.x



11. Can I attach a client script in a user event script?

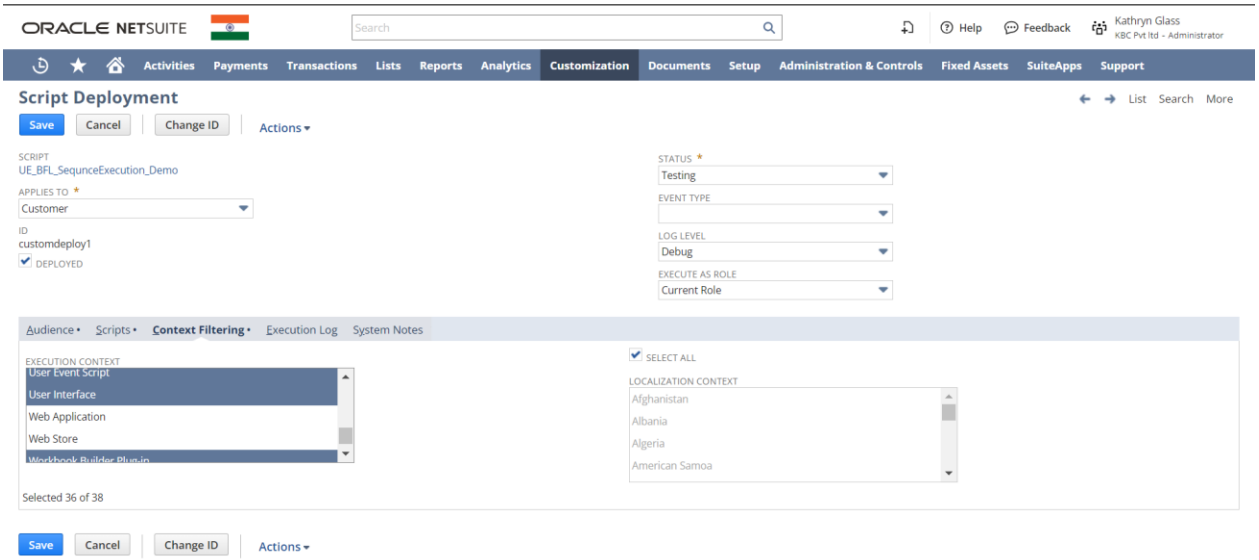
Ans.

- Yes, we can attach a client in a user event script.
- We have API in form module which could be used to attach client script in event script.
  1. Form.clientScriptFileId = ‘The internal file ID of client script file’
  2. Form.clientScriptModulePath = ‘The relative path of client script file in file cabinet’

12. How can we get the execution context?

Ans.

- Navigation for execution context: Customization > Script > New > Upload user event script file with .js extension > Create script record > Enter file name and Id > Save > Deploy Script > Context filtering > Execution context.



- Execution context can also get form Suite script 2.x module
- We can use runtime module to use execution context.
- Runtime.executionContext This event triggers the current script.
- Execution contexts provide information about how a script is triggered to execute. For example, a script can be triggered in response to an action in the NetSuite application, or an action occurring in another context, such as a web services integration.
- We can use execution context filtering to ensure that scripts are triggered only when necessary.

- We can use the runtime.executionContext property to determine the execution context for a script and choose different logic depending on the context.
- Consider a script that applies to customer records and uses the beforeLoad(context) entry point. We may not want to execute this script when the entry point is triggered in response to a REST web services request.
- if (runtime.executionContext === runtime.ContextType.REST_WEBSERVICES) {
  return;
  }

13. Is there any API that can't be used in a user event script?

Ans.

API that can't be used in a user event script:

- N/portlet
- N/dialog
- N/message

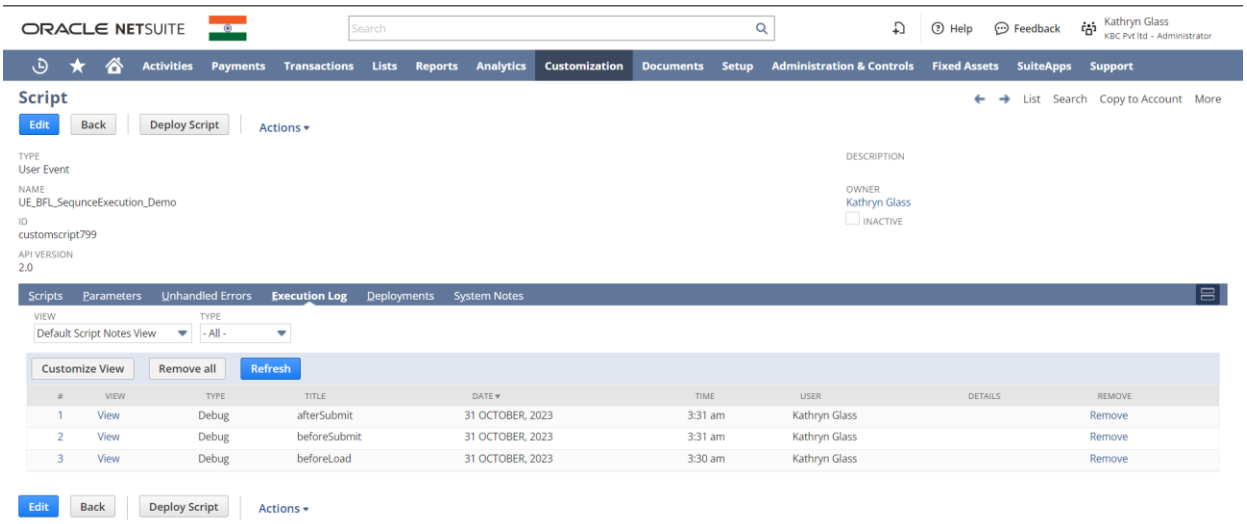
14. Use log.debug to notice the type of three events that is beforeLoad,beforeSubmit,afterSubmit in the user event script.

Ans.

- Script using log.debug for three user events:

```
/**
 *@NApiVersion 2.x
 *@NScriptType UserEventScript
 */

define(["N/record"], function (record) {
  function mybeforeLoad(context) {
    log.debug({
    title: "beforeLoad "
    });

  }
  function myBeforeSubmit(context) {
    log.debug({
    title: "beforeSubmit "
    });
  }

  function myAfterSubmit(context) {
    log.debug({
    title: "afterSubmit "
    });
  }

  return {
    beforeLoad: mybeforeLoad,
    beforeSubmit : myBeforeSubmit,
    afterSubmit: myAfterSubmit

  };
});
```

- Sequence of execution log of user events: beforeLoad > beforeSubmit > afterSubmit



15. Related to the newRecord and oldRecord.

Ans.

newRecord:

- This object represents the state of the record after the changes have been made. We can use it to access the current values of fields in the record.
- In user events we have newRecord in beforeLoad, beforeSubmit, afterSubmit as a parameter

oldRecord:

- This object represents the state of the record before the changes were made. We can use it to access the values of fields in the record before any modifications.
- In user events we have oldRecord in beforeSubmit, afterSubmit as a parameter

These objects are particularly useful when we want to compare the old and new values of a field to implement custom validation, trigger specific actions when a field changes, or track changes to a record over time

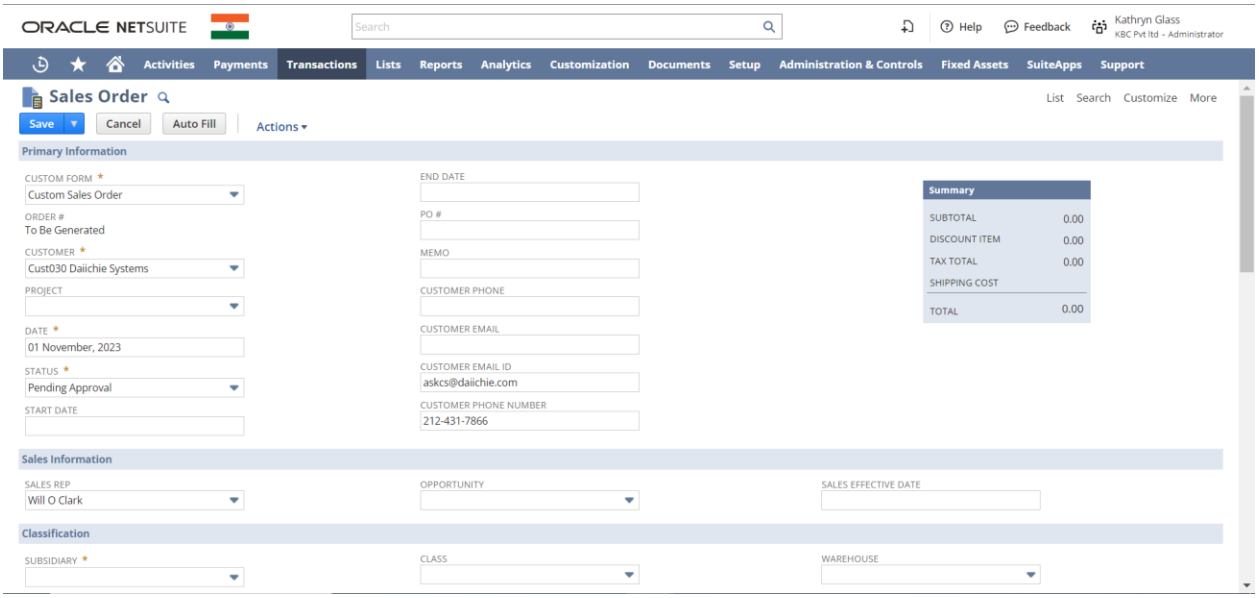
16. Email and Phone number of customer should be populated on the transaction before the loading of the record.

Ans.

- Here we will use beforeLoad() to autofill the customer email and phone number in sales order
- We will use lookpField() to fetch customer email and phone number from customer record.

```
/**
 * @NApiVersion 2.x
 * @NScriptType UserEventScript
 */
define(['N/record', 'N/search'],  function(record, search){
        function beforeLoad(scriptContext){

            var o_recObj = scriptContext.newRecord;
            var i_customerId = 792; //o_recObj.getValue({fieldId: 'entity'});
            log.debug('customer Id = ', i_customerId);

            var o_custObj = search.lookupFields({type: 'customer', id: i_customerId, columns: ['entity' ,'email',
            log.debug('email = ', o_custObj.email);
            log.debug('email = ', o_custObj.phone);
            o_recObj.setValue({fieldId: 'entity', value: 792 })
            o_recObj.setValue({fieldId: 'custbody1', value: o_custObj.email})
            o_recObj.setValue({fieldId: 'custbody2', value: o_custObj.phone})

        }

        return {beforeLoad :beforeLoad}

    });
```

- Here customer Email and phone number has been auto filled.



17. Like the above one but now it should get populated after the record

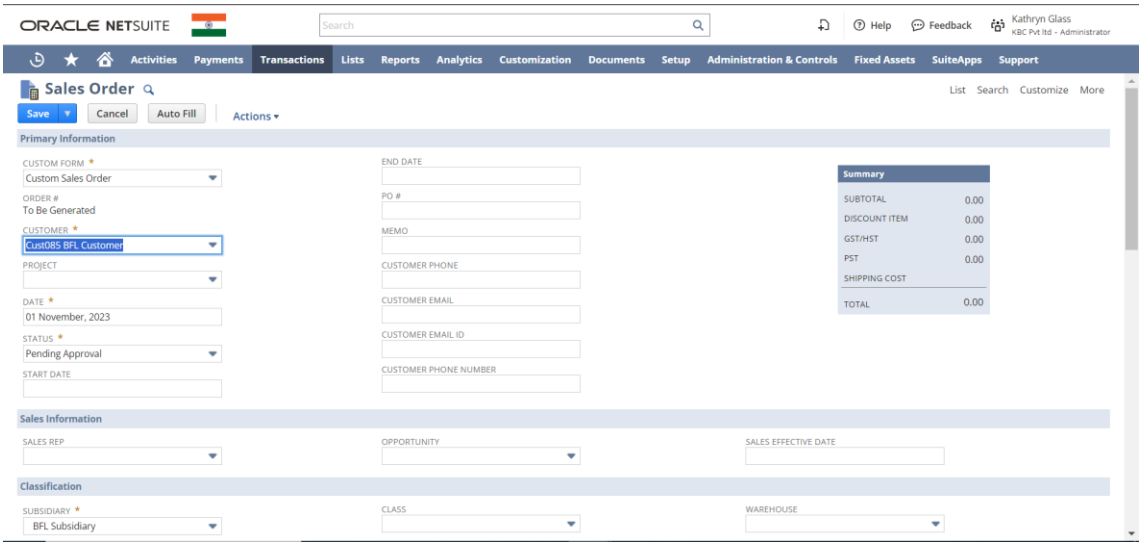
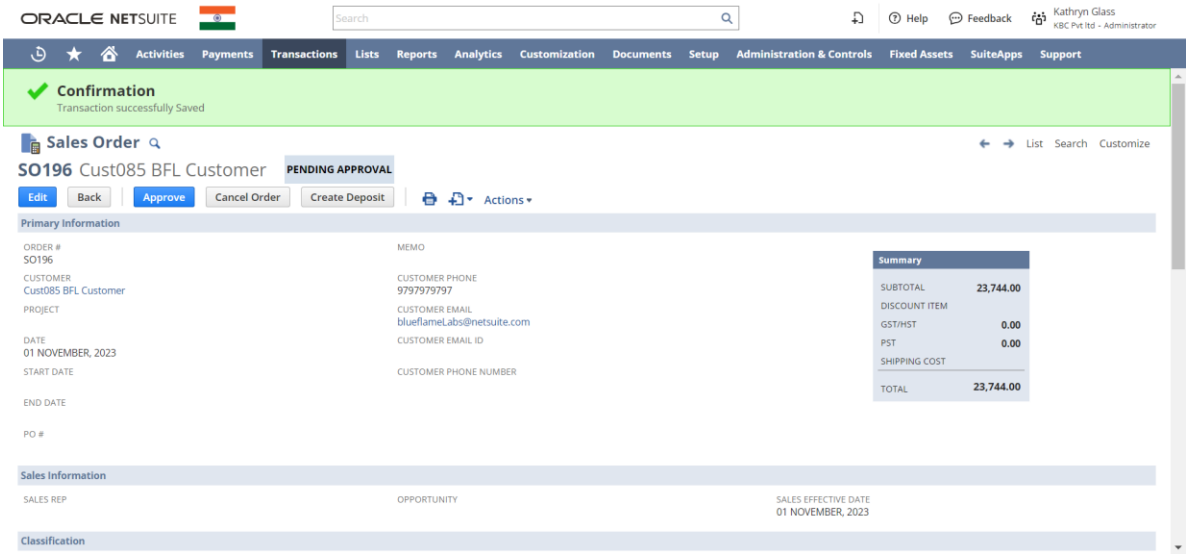
Ans.

- In user event script we will use getValue() to take id of customer from sales order
- Using record.load() we will fetch email and phone form customer record
- This record will be saved to phone and email field of sales order

```
/**
 * @NApiVersion 2.x
 * @NScriptType UserEventScript
 */
define(['N/record', 'N/search'],  function(record, search){
        function beforeSubmit(scriptContext){

            var objRecordSales = scriptContext.newRecord;

            var cust_id = objRecordSales.getValue({
                fieldId: "entity"
            });

            var objRecordCustomer = record.load({
            type: record.Type.CUSTOMER,
            id: cust_id
            });

            var cust_phone = objRecordCustomer.getValue({
                fieldId: "phone"
            });
            var cust_email = objRecordCustomer.getValue({
                fieldId: "email"
            });
```

```
            objRecordSales.setValue({
                fieldId: "custbody_customer_phone",
                value: cust_phone,
            });

            objRecordSales.setValue({
                fieldId: "custbody_cutomer_email",
                value: cust_email,
            });



            log.debug("objRecordSales", objRecordSales );



        }

        return {beforeSubmit :beforeSubmit}

    });
```

- We will only select Customer name and item for sales order and save



- After saved the sales order, customer phone number and email will get reflect in fields automatically